BDH-GPU for Searchless Chess

Proposal Presentation

Antoni Czapski

Project: Deep Learning

March 10, 2026

Problem: Searchless Chess

Input: Chess board position (FEN)
Output: Position score $\in [-1, 1]$
Task: Supervised regression

No minimax. No alpha-beta. No MCTS.
Pure neural pattern recognition.

My angle: Train a novel post-transformer architecture (BDH-GPU) and compare against ViT.

Prior Art

Model	Params	ELO
DeepMind '24	270M	2895
ViT-Med*	9.5M	1960
ViT-Small*	2.6M	1817
ResNet-XL*	24.7M	1719

* Grzyb, 2025 — open dataset & models

Why BDH-GPU?

BDH in one sentence

Recurrent linear-attention model with Hebbian fast-state updates and shared weights across layers.

Three reasons it fits chess:

1. **Relational state** — piece interactions (pins, forks, batteries) stored as neuron-pair synaptic weights
2. **Iterative reasoning** — same layer applied K times = "thinking longer" about hard positions
3. **Sparse activations** — only relevant patterns fire, natural match for chess motifs

No prior work has applied BDH to chess or any game domain.

Dataset

Lichess-Stockfish-Normalized

`huggingface.co/datasets/mateuszgrzyb/lichess-stockfish-normalized`

Positions	316M
Format	Parquet (10 shards)
License	CC BY 4.0
I use	∼50–100M subset

Split: 90% / 5% / 5% (seed 2001)

Encoding

FEN → 8×8×12 binary tensor

64 squares × 12 piece channels
Always-white perspective

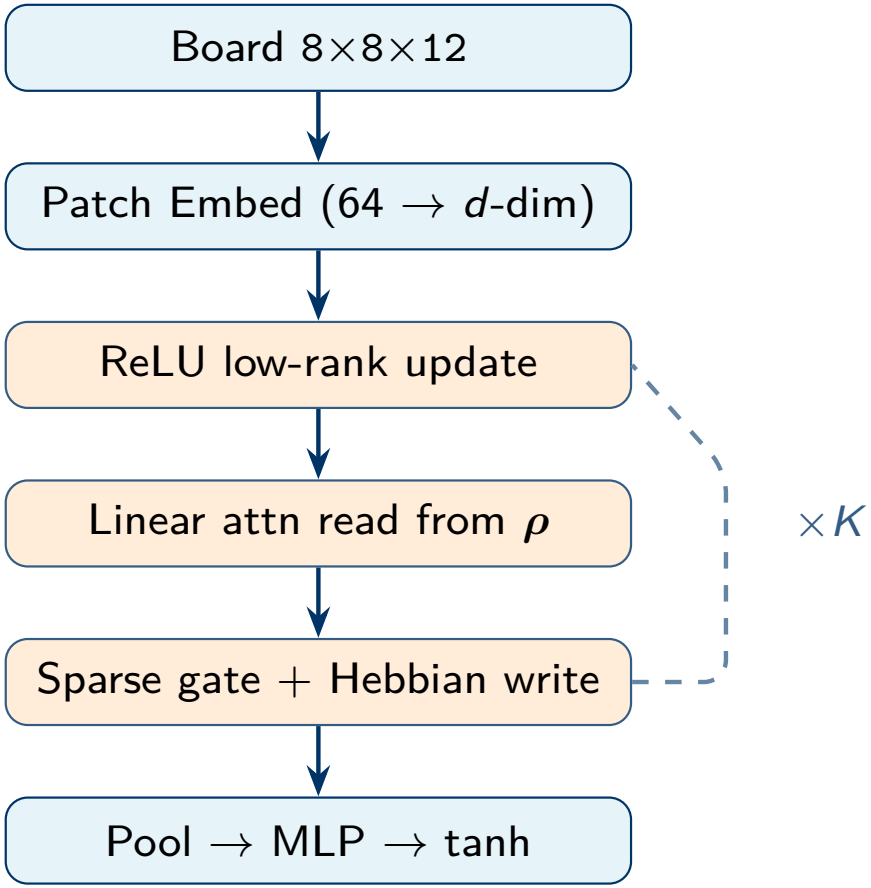
Pipeline

HuggingFace download → encode → split → TFRecord streaming

Key

Proven dataset — already produced 1960 ELO models. Direct comparability.

Architecture & Evaluation



Model variants:

	Params	K
BDH-Tiny	1.6M	4
BDH-Small	6.3M	8
BDH-Medium	12.6M	12

Baselines: MLP (∼2M), ViT-Small (2.6M)

Metrics:

- MSE (training loss)
- ELO estimation via 1200 Lichess puzzles (50% accuracy threshold)

Engineering & DevOps

uv + Config-driven training

```
uv sync
uv run python scripts/training/train.py \
  --config configs/train_bdh_small.yaml
```

CI/CD (GitHub Actions)

- `ruff` — lint + format
- `pytest` — 5+ test cases
- On every push & PR

Tests (pytest)

- Data loading & encoding
- BDH forward pass + shapes
- Loss sanity + training smoke

Reproducibility

- Seeds in YAML → `tf`, `np`, `random`
- Fixed splits (saved to JSON)
- `uv.lock` pinned deps
- `ModelCheckpoint` best model

W&B (≥7 experiments)

1. MLP baseline
2. ViT-Small baseline
3. BDH-Tiny
4. BDH-Small
5. Depth ablation (K=4,8,12)
6. BDH-Medium
7. Adaptive depth

Compute, Timeline & Risks

100h on NVIDIA A100:

Task	Hours
Data pipeline + baselines	9h
BDH-Tiny (iteration)	5h
BDH-Small (main)	20h
Depth ablation	15h
BDH-Medium	40h
Eval + buffer	11h
Total	**100h**

BDH-Medium is expendable if behind schedule.

Key risks:

BDH underperforms ViT

Still valid as architecture comparison. Strong baselines ensure contribution.

No reference implementation

Start tiny, unit-test everything, verify gradients early.

Meeting roadmap:

4	Repo + CI green
5	EDA + baselines
6–7	BDH training + W&B
8–9	Ablations + error analysis
10–11	Scale-up + polish
12	Final presentation

Summary

Problem	Searchless chess position evaluation
Dataset	316M positions (HuggingFace, CC BY 4.0)
Architecture	BDH-GPU — post-transformer with Hebbian state
Metric	MSE + ELO via 1200 puzzle benchmark
Engineering	`uv` + CI + `ruff` + `pytest` + W&B + YAML
Compute	100h on NVIDIA A100

Questions?